The DAPT × Transformer Topology pipeline, start to end

This document explains how the whole project is built, file by file and decision by decision. It assumes no prior knowledge of the project. If you only have ten minutes, read sections 1, 2 and 4. If you will be questioned on the work, read everything, then read section 10.

Status of the facts in this document: the code behaviour comes from reading the files in `src/` and `scripts/` as of commit `c6173b6`. Numbers marked *(plan)* come from `WEEK3_PLAN_3DAYS.md` and were not re-derived here. Nothing in this document comes from a real Colab run; the real runs were still in progress when it was written (2026-10-14).

---

1. What the project asks, in plain words

Background terms you need.

Term	Meaning
ABSA (aspect-based sentiment analysis)	Given a sentence and one *aspect* (a word or phrase in it, such as "pasta"), say whether the sentiment about that aspect is negative, neutral or positive. The same sentence can have several aspects with different answers.
MAMS-ATSA	The benchmark we evaluate on. Restaurant review sentences from Citysearch, built so that every sentence has at least two aspects with *different* sentiments. A model that ignores the aspect scores badly.
Pretrained model	A neural network (here a Transformer) already trained on huge general text. We start from public ones.
DAPT (domain-adaptive pretraining)	Continuing the pretraining of a model, with its original objective, on text from the target domain, *before* fine-tuning. The idea comes from Gururangan et al. 2020.
Fine-tuning	Training the model on the labelled task data (MAMS) so it solves the task.
Topology	The shape of the Transformer: encoder-only (RoBERTa), decoder-only (GPT-2), or encoder-decoder (T5).
MLM / CLM / span corruption	The three pretraining objectives, one per topology. MLM hides random tokens and predicts them. CLM predicts the next token. Span corruption hides contiguous spans and regenerates them in a decoder.
Macro-F1	The F1 score computed per class (negative, neutral, positive) and averaged with equal weight. It does not let the biggest class dominate.
Seed	The number that fixes the random choices of a run (data order, initial values of new layers, dropout). Several seeds show how much a result depends on luck.

The research question. Does DAPT help ABSA, and does *how much* it helps depend on the Transformer topology?

The headline number is not accuracy. It is **Δ_DAPT**: for each architecture, the test macro-F1 of the model that went through DAPT minus the test macro-F1 of the same model without it. Because the paper compares those Deltas across architectures, absolute scores matter much less than keeping everything else identical. That single idea drives most design decisions below.

2. Experimental design

Four models, each fine-tuned two ways, three seeds each:

Key	Pretrained checkpoint	Topology	DAPT objective	Parameters
`roberta`	`roberta-base`	encoder-only	MLM	about 125M
`gpt2`	`gpt2` (small)	decoder-only	CLM	about 124M
`t5`	`t5-base`	encoder-decoder	span corruption	about 220M
`t5small`	`t5-small`	encoder-decoder	span corruption	about 60M

`t5small` is a **scale control**. T5-base is much larger than the other two, so a T5 result could be blamed on size. T5-small sits on the other side of the others and lets the paper say whether size or topology explains the pattern.

- Fine-tuning initializations: `base` (the stock checkpoint) and `dapt` (the checkpoint after our DAPT).
- Seeds: 13, 42, 1337.
- Total: 4 architectures × 2 initializations × 3 seeds = **24 fine-tuning runs**, plus **4 DAPT runs** (one per architecture). Older text in `README.md` says 18 runs; that was before `t5small` joined.

What is held constant on purpose.

- The DAPT corpus and the token budget: every model sees 50M tokens of Yelp restaurant text.
- The fine-tuning recipe between `base` and `dapt` of one architecture: byte-for-byte the same hyperparameters, from the single file `src/config.py`.
- The way predictions are made: every model picks the best of exactly three label strings (section 5.8).
- The train, validation and test splits (the official MAMS ones).

What is allowed to differ, and why.

- Parameter count (see "Parameter matching is dropped" in section 9).
- Fine-tuning learning rate per architecture (section 9).
- The pretraining objective (it is part of the architecture, section 9).

3. Repository map

```
src/config.py          Single source of truth: hyperparameters, paths, token budget, seeds.
src/build_corpus.py    Yelp text -> cleaned, deduplicated corpus -> packed token blocks.
src/collators.py       The three DAPT objectives (MLM, CLM, T5 span corruption).
src/dapt.py            DAPT training loop with checkpoint/resume and the before/after loss.
src/data_mams.py       Reads the MAMS XML into (sentence, aspect, label) rows.
src/absa_models.py     The three task heads (RoBERTa, GPT-2, T5) and constrained scoring.
src/train_absa.py      The ONE fine-tuning script; also runs the DAPT guard.
src/aggregate.py       Builds the results table, Deltas and significance tests.
scripts/check_labels.py    Prints how many tokens each label string is for GPT-2/T5.
scripts/check_lengths.py   Counts MAMS prompts that would be truncated for GPT-2/T5.
data/                  Git-ignored. mams/, yelp_clean.txt, packed/ (lives on Drive).
checkpoints/           Git-ignored. <arch>_dapt/ (model, report, resume state).
runs/                  Git-ignored. One JSON per fine-tuning run.
logs/                  Git-ignored. One log per command.
```

Run every command as `python -m src.<module>` from the repository root with the environment variable `PYTHONPATH=src`. The modules import each other as `from config import ...`, which only resolves with that path set.

Where things live. Git holds code only. The Colab local disk is wiped when a session ends, so Colab mounts a shared Google Drive folder and symlinks `data`, `checkpoints`, `runs` and `logs` to it. Every teammate's notebook sees the same data and results.

4. The pipeline at a glance

```
 Yelp Open Dataset (review.json + business.json)
        │  build_corpus --stage text      (filter restaurants/food, clean, dedupe, 650k reviews)
        ▼
 data/yelp_clean.txt   (one review per line)
        │  build_corpus --stage pack --arch X   (tokenize with X's tokenizer, chop into 256-token
blocks)
        ▼
 data/packed/X_train.npy (195,312 blocks = 49,999,872 tokens)   X_heldout.npy (2,000 blocks)
        │  dapt --arch X                  (continue pretraining; log held-out loss before and
after)
        ▼
 checkpoints/X_dapt/  (model, tokenizer, dapt_report.json)
        │  Gate 2: loss dropped? tokens ≈ 50M? no NaN?  (code guard + C's check)
        ▼
 train_absa --arch X --init dapt --seed S   ─┐
 train_absa --arch X --init base --seed S   ─┤   (MAMS train; pick best epoch on MAMS val; score
MAMS test once)
        ▼                                    │
 runs/X_{base,dapt}_sS.json  ◄───────────────┘
        │  aggregate
        ▼
 Table: mean ± std per cell, Δ_DAPT per architecture, paired t-tests, cross-architecture test
```

Baselines (`--init base`) do not depend on DAPT, so they run first. The ordering in `README.md` is: corpus, baselines, DAPT, DAPT fine-tunes, aggregate.

5. Stage by stage

5.1 `src/config.py`: one place for every number

All hyperparameters live here and nowhere else. The reason is the experimental design: if the `base` and `dapt` runs of an architecture differed by even one hyperparameter, the Delta would partly measure that difference. Putting everything in one file, with no per-run overrides, makes "identical recipe" checkable by reading one file.

Key values:

Name	Value	Meaning
`DAPT_TOKEN_BUDGET`	50,000,000	Tokens each model trains on during DAPT.
`DAPT_BLOCK_LEN`	256	Length of one training block. Reviews are short; 512 would waste compute.
`dapt_bs`	32	Blocks per optimizer step, so 8,192 tokens per step.

Name	Value	Meaning
`dapt_lr`	5e-5	DAPT learning rate, same for all four models.
fine-tune `epochs` / `train_bs` / `max_len`	5 / 32 / 128	Same for all four models.
fine-tune `warmup_ratio` / `weight_decay` / `grad_clip`	0.1 / 0.01 / 1.0	Same for all four.
fine-tune `lr`	roberta 2e-5, gpt2 5e-5, t5 1e-4, t5small 3e-4	Differs per architecture (section 9).
`SEEDS`	13, 42, 1337	

Smoke mode. The environment variable `SMOKE=1` switches the token budget to 320,000 and redirects every path to a `_smoke` twin (`checkpoints_smoke`, `runs_smoke`, `data/packed_smoke`). This lets the team test the whole pipeline in minutes without touching real outputs, and without editing `config.py`. Smoke runs must never be mixed into the real table: `aggregate.py` warns if they are.

Two environment variables can still change the recipe silently: `DAPT_BS` (DAPT batch size, default 32, meant for small GPUs) and `DAPT_SAVE_EVERY` (checkpoint interval). They are not recorded in the output files (see section 10).

5.2 Corpus: `build_corpus.py --stage text`

Input. The official Yelp Open Dataset: `yelp_academic_dataset_business.json` and `yelp_academic_dataset_review.json`.

Why Yelp restaurants and not Amazon. MAMS is built from restaurant reviews. DAPT only helps when the adaptation text is close in *domain* to the task text (Gururangan et al.). Amazon reviews talk about shipping and batteries; restaurant reviews talk about service and portions. "Reviews" is a genre, not a domain. An early draft of the methodology section said Amazon and must be corrected.

Steps, in order (`iter_official`, `clean`, `build_text`):

1. Read the business file and keep every business whose category string contains the substring "Restaurants" or "Food" (64,616 businesses *(plan)*). It is a substring test, so it also matches unrelated categories that contain those words.
2. Stream the review file in file order and keep reviews of those businesses.
3. Clean each review:
4. replace URLs by a space and the literal two-character sequence `\n` by a space;
5. collapse all whitespace to single spaces;
6. drop reviews shorter than 40 characters ("Great!" teaches nothing);
7. **truncate** reviews longer than 4,000 characters (they are cut, not dropped);
8. drop reviews where fewer than 85% of the characters are letters or spaces (menus, phone numbers, star spam).
9. Deduplicate: an MD5 hash of the first 200 characters. A review whose first 200 characters were already seen is skipped. This only catches near-duplicates that share a prefix.
10. Stop after 650,000 kept reviews. The corpus is therefore *the first 650,000 qualifying reviews in file order*, not a random sample, and the paper must say so. Average length is about 537 characters

(plan).
11. Write one review per line to `data/yelp_clean.txt`.

The 650,000 figure is a safety margin: roughly 400,000 reviews are needed for 50M tokens, and the margin covers the held-out set and tokenizer differences.

5.3 Packing: `build_corpus.py --stage pack --arch X`

Each model has its own tokenizer, so the corpus is tokenized and packed once per architecture (`t5` and `t5small` share the same vocabulary but are packed separately, which is harmless).

Why packing. If every review were padded to 256 tokens, about half of the "50M tokens" would be padding and the number in the paper would be false. Instead the code concatenates the tokenized reviews, putting an end-of-sequence token between them, and cuts the stream into blocks of exactly 256 real tokens. Consequences worth knowing:

- A block can contain the end of one review and the start of the next. The model's attention crosses review boundaries. This is the standard way to pack, and it is a simplification.
- The budget is counted in *real tokens seen*, not in reviews. Different tokenizers split text differently, so they use different numbers of reviews to reach 50M tokens (about 414,813 reviews for RoBERTa and GPT-2, about 378,280 for T5 *(plan)*). The paper equalizes tokens, not reviews.

Train and held-out split. The code reads until it has 50,000,000 + 2,000 × 256 tokens. The first 195,312 blocks (49,999,872 tokens) become `X_train.npy`; the next 2,000 blocks become `X_heldout.npy`. The held-out blocks are the continuation of the same review stream, so they come from later reviews than the training blocks. Blocks are stored as 32-bit integers, which is why each train file is 195,312 × 256 × 4 + 128 header bytes = 199,999,616 bytes.

5.4 The three DAPT objectives: `collators.py`

A *collator* turns a list of raw blocks into one training batch, including the corrupted input and the labels. Each objective belongs to its topology.

MLM for RoBERTa (`MLMCollator`). Select each non-special token with probability 15%. Of the selected tokens, 80% are replaced by `<mask>`, 10% by a random token and 10% are left unchanged. The loss is computed only at the selected positions (labels elsewhere are -100, which PyTorch ignores). Masks are drawn fresh for every batch (*dynamic masking*). Special tokens, including the end-of-sequence separators, are never selected. This is the standard BERT/RoBERTa recipe.

CLM for GPT-2 (`CLMCollator`). Labels equal inputs; the Hugging Face model shifts them internally so each position predicts the next token. Loss is on essentially every position (about 255 per block).

Span corruption for T5 (`SpanCorruptionCollator`). Reimplemented from Raffel et al. because Hugging Face ships it only in a Flax example. With noise density 0.15 and mean span length 3, a 256-token block has about `round(256 × 0.15) = 38` noised tokens in about `round(38 / 3) = 13` spans. The code splits the 38 noised tokens into 13 random positive-sized runs, splits the 218 kept tokens into 13 runs, and alternates keep, noise, keep, noise... The encoder input is the kept tokens with each noised span replaced by one sentinel token (`<extra_id_0>`, `<extra_id_1>`, ...), about 232 tokens including the end token. The decoder target is each sentinel followed by the original tokens of that span, ending with the end token,

about 52 tokens, truncated at 128. One small difference from the original paper: the target has no trailing extra sentinel before the end token.

Why the three are not equal, and why that is stated openly. MLM supervises about 38 tokens per block, CLM about 255, T5 about 52. The objective is part of what makes the architecture the architecture, so it cannot be normalized away. The project controls the one quantity it can (source tokens processed, 50M) and lists the density mismatch as a limitation.

5.5 DAPT training: `dapt.py`

Data loading (`blocks_loader`). Loads the `.npy` blocks, shuffles them with one fixed permutation (seed 0), and serves batches of 32 with the last incomplete batch dropped. So an epoch has 195,312 // 32 = **6,103 steps**, and DAPT is exactly **one pass** over the data. Because the order is fixed, a resumed run can skip exactly the batches already trained on.

The loop.

1. Load the model for the objective: `AutoModelForMaskedLM`, `AutoModelForCausalLM` or `AutoModelForSeq2SeqLM`, from the stock checkpoint (or from our own checkpoint when resuming).
2. Measure the held-out loss *before* training (skipped when resuming; the saved value is reused).
3. Build AdamW (lr 5e-5, weight decay 0.01) and a linear schedule: warmup for 6% of the steps (366), then linear decay to zero.
4. For each batch: forward pass under bf16 autocast, backward, clip the gradient norm to 1.0, optimizer step, scheduler step.
5. Every 100 steps print loss and tokens seen; every 1,000 steps save a checkpoint.
6. At the end save the model and tokenizer, measure the held-out loss *after*, write `dapt_report.json`, and delete the resume files.

Precision. bf16 autocast with fp32 master weights; no loss scaler is needed because bf16 has the range of fp32. bf16 is used for every model because T5 overflows in fp16, and one precision keeps DAPT uniform. (Fine-tuning runs in plain fp32, see 5.8.)

Checkpoint and resume. Colab can disconnect at any time, and T5-base DAPT takes about 90 minutes *(README)*, so DAPT is resumable. Every 1,000 steps it saves three things in a fixed order: the optimizer-and-scheduler file, then the model and tokenizer, then `state.json` (step and the original `loss_before`), written *last*. On restart the code compares the step stored in the optimizer file with the step in `state.json`. If they disagree, a crash happened in the middle of a save, and the run refuses to continue rather than mix files from two different moments (the message says to rerun with `--no-resume`). When resuming, the log shows `RESUMING from step N/6103`. The masks drawn after a resume differ from an uninterrupted run, so a resumed run is correct but not bit-for-bit identical.

The held-out loss (`heldout_loss`). Computed on the 2,000 held-out blocks (62 batches, the last partial batch dropped), in eval mode, with the mask random generator forked and fixed to seed 0. So "before" and "after" score exactly the same corruptions, and the difference is due to the weights alone. The number is the mean of the per-batch mean losses.

The report. `dapt_report.json` holds `step`, `loss_before`, `loss_after` and `tokens` (computed as `step × batch size × 256`, so 6,103 × 32 × 256 = 49,995,776). The report is deleted at the start of every run, so a report on disk always describes a *finished* run.

5.6 MAMS loading: `data_mams.py`

Reads `train.xml`, `val.xml`, `test.xml`. Each XML sentence contains several aspect terms; the loader emits **one row per aspect term** with `sentence`, `aspect` and a numeric label (negative 0, neutral 1, positive 2). Terms labelled anything else (such as `conflict`) are skipped. Sizes: train 11,186 rows (4,297 sentences), validation 1,332 rows (500 sentences), test 1,336 rows (500 sentences). A check run for this document found no sentence shared between any two splits, and no duplicated (sentence, aspect) pair inside a split.

5.7 Fine-tuning data flow in one sentence

For every run: build the model from `base` or from `checkpoints/X_dapt`, train 5 epochs on MAMS train, evaluate on MAMS validation after each epoch, keep the weights of the epoch with the best validation macro-F1, load those weights, score MAMS test once, save everything in a JSON.

5.8 Task heads: `absa_models.py` (the fairness rule)

The three models have different shapes, but all must be scored the same way: choose the best of exactly three labels. If RoBERTa used a clean 3-way softmax while GPT-2 and T5 were allowed to write free text, the comparison would measure decoding strategy and format errors rather than topology. So the generative models **never generate**. At test time they score each of the three label strings and take the argmax.

RoBERTa (`RobertaABSA`). The sentence and aspect are encoded as a pair, `<s> sentence </s></s> aspect </s>`, truncated to 128 tokens. A standard classification head (dropout, linear 768→768, tanh, dropout, linear 768→3) sits on the first token. Loss: 3-way cross-entropy. When starting from a DAPT checkpoint, the pretraining output layer is discarded and this head is newly initialized.

GPT-2 (`GPT2ABSA`). The input is the text `Review: {sentence}\nAspect: {aspect}\nSentiment:` followed by one of `negative`, `neutral` or `positive`. Training loss is on the label tokens only (the prompt is masked with -100). At test time, the code builds the sequence three times, once per candidate label, sums the log-probability of the label tokens, and takes the argmax. Padding reuses the end-of-text token (GPT-2 has none). The prompt is truncated to 124 tokens (`max_len - 4`), and truncation cuts from the **right**, so a very long prompt would lose the aspect and the `Sentiment:` cue. `scripts/check_lengths.py` counts how many MAMS prompts exceed 124 tokens.

T5 (`T5ABSA`, used for both `t5` and `t5small`). The input is `absa sentence: {sentence} aspect: {aspect}`; the target is the label word followed by the end token, trained with teacher forcing. At test time the code scores each of the three label sequences by the summed log-probability of its tokens and takes the argmax.

Known bias of this scoring. A summed log-probability favours labels that are fewer tokens long. `scripts/check_labels.py` prints the token count of each label for GPT-2 and T5. If the three labels have different lengths, that is a small bias to report; after fine-tuning it should be small because the model is trained on exactly these strings.

5.9 The fine-tuning script: `train_absa.py`

Arguments. `--arch {roberta,gpt2,t5,t5small}`, `--init {base,dapt}`, `--seed N`.

The DAPT guard (Gate 2 in code). With `--init dapt`, the script refuses to start unless `checkpoints/X_dapt/dapt_report.json` exists, its `tokens` is at least 99% of the budget, and `loss_after < loss_before`. This makes it impossible to fine-tune from a missing, partial or ineffective DAPT checkpoint by accident.

Provenance. Before training the script records the git commit (`git describe --always --dirty`, so uncommitted changes show up as `-dirty`) and fails immediately if git is unavailable.

Optimization. AdamW with the architecture's learning rate and weight decay 0.01 applied to all parameters, a linear schedule with 10% warmup, gradient clipping at 1.0, batch size 32, 5 epochs, fp32 (no autocast). The scheduler is built for `(11,186 // 32) × 5 = 1,745` steps, but the loop runs 350 batches per epoch (the last has 18 rows), so the final 5 optimizer steps run at learning rate 0. This is identical for `base` and `dapt` and has no practical effect.

Seeding. `set_seed` seeds Python, NumPy and torch (CPU and CUDA) before the model is built. That fixes the shuffling of the training rows and the random initialization of any new layer. The same seed therefore gives `base` and `dapt` the same training-row order, which is what makes pairing by seed meaningful. GPU kernels are not forced to be deterministic, so tiny numeric differences between reruns remain possible.

Model selection. After each epoch the script computes validation accuracy and macro-F1. If the macro-F1 is the best so far, it copies the weights to CPU memory. After the last epoch it loads the best weights and evaluates the test set **once**. The test set is never used for any decision. This is the rule that keeps the Deltas honest.

Output. `runs/X_{init}_s{seed}.json` contains the validation F1 of the chosen epoch, test accuracy and macro-F1, the full list of test `preds` and `gold`, parameter count, git commit, GPU name, `transformers` version, `smoke` flag and `dapt_tokens`. Keeping `preds` and `gold` allows error analysis and bootstrap tests later without any GPU.

5.10 Aggregation and statistics: `aggregate.py`

1. **Load** every JSON in `runs/` and group by (architecture, initialization).
2. **Provenance check.** Warns about mixed git commits, mixed GPU types, a mix of smoke and real runs, dirty or unknown commits, and `dapt` runs fine-tuned from a DAPT run that saw fewer than 99% of the token budget. It does *not* check the `transformers` version.
3. **Table.** For each cell: mean ± sample standard deviation (n−1) of accuracy and macro-F1, in percent.
4. **Δ_DAPT per architecture.** For each seed present in both `base` and `dapt`, take `dapt F1 − base F1`, then report the mean and standard deviation of those paired differences and a **paired t-test** across seeds. Pairing by seed is justified because the same seed fixes the same training-row order and head initialization.
5. **Does topology modulate the benefit?** For every pair of architectures, an independent two-sample t-test between their sets of three Deltas. This asks whether the difference between Deltas is distinguishable from noise: a claim about differences of differences.

The code itself warns that with three seeds these tests are badly underpowered (a t-test on three values has two degrees of freedom). If Deltas are close, the honest conclusion is "not distinguishable", not "X wins".

6. The two gates

Gate 1, baseline sanity. Before any DAPT result is believed, the RoBERTa baseline must reach roughly 82–84% accuracy on MAMS-ATSA test *(README)*. If it does not, the fine-tuning recipe is broken and every Delta would measure a bug. The team's RoBERTa baseline is 85.05 *(plan)*, which is above the README's range and above BERT's 82.22 and CapsNet-BERT's 83.39 reported in the MAMS paper; the comparison must be accuracy against accuracy, and the paper used 5 runs while we use 3.

Gate 2, did DAPT happen? A model passes when its held-out loss dropped, no value is `nan`, and the token count is within 1% of 50M. Two implementations exist:

- the guard in `train_absa.py`, which enforces the first and third conditions automatically;
- person C's check, which also catches `nan` and must report PASS *before* anyone fine-tunes that model (team rule).

The drop proves only that DAPT changed the model on Yelp text. It does not prove DAPT helps ABSA. That is the purpose of the experiment.

7. Reproducibility and provenance

- **Pinned environment.** `requirements.txt` fixes transformers 4.46.3, numpy 2.1.3, datasets, accelerate, scikit-learn, scipy and sentencepiece. Colab runtime 2025.07 (Python 3.11) is required because newer runtimes cannot build the pinned wheels.
- **One GPU type, one commit.** Every run records `gpu_name` and `git_commit`; `aggregate.py` warns when a table mixes them. All real runs use an L4.
- **Logs.** Every command is piped through `tee logs/<name>.log`.
- **No code changes during real runs** to `train_absa.py`, `absa_models.py`, `dapt.py` or `config.py` unless the whole team agrees.
- **The baselines were run on commit `0ffc6e8`; DAPT runs use a later commit.** The only files that differ are `.gitignore` and `requirements.txt` (checked with `git diff freeze-ft-v1 HEAD`), so no training code changed. `aggregate.py` will print a mixed-commit warning that the paper explains.

8. How the work is scheduled (week 3)

Three people: A (results and writing), B (all GPU runs), C (Gate 2 checks, Discussion and Limitations). GPU work all happens on B's single L4 on Day 1 (about 5–6 hours *(plan)*: roughly 3.3 hours of DAPT and 1.5 hours of fine-tuning). Each model is its own pipeline: DAPT, Gate 2 by C, then three fine-tunes. Day 2 is a buffer plus the final table. Day 3 finishes the paper. Extra seeds and error analysis are out of scope. See `WEEK3_PLAN_3DAYS.md`.

9. Design decisions and why

Decision	Reason	Cost or caveat
Yelp restaurants, not Amazon	Domain proximity to MAMS is what makes DAPT work.	Corpus is the first 650k qualifying reviews, not a random sample.

Decision	Reason	Cost or caveat
Match on DAPT tokens (50M), not parameters	RoBERTa 125M, GPT-2 124M, T5-base 220M differ by 76%; "approximately constant" is not defensible.	Report parameter counts; use `t5small` as scale control.
Pack the corpus into 256-token blocks	Padding would waste about half the budget and make "50M tokens" false.	Attention crosses review boundaries.
Constrained scoring for GPT-2 and T5	Free generation would measure decoding and format errors, not topology.	Summed log-probability is length-sensitive.
Fine-tuning learning rate differs per architecture	T5 needs about 1e-4, RoBERTa about 2e-5; one rate would cripple a model and be reported as an architecture effect.	The rate is identical between `base` and `dapt` of one architecture, which is the contrast that matters. Not re-tuned for DAPT models.
One DAPT learning rate (5e-5) for all four	Keeps the DAPT recipe uniform.	Low for T5 compared with its usual training; may under-adapt T5.
Keep each architecture's native objective	The objective is part of the architecture.	Supervised tokens per block differ (about 38, 255 and 52). Stated in Limitations.
One fine-tuning script for all cells	Guarantees the only change between `base` and `dapt` is the initialization.	None.
Select the epoch on validation, test once	Prevents tuning against test.	Validation has only 1,332 rows, so epoch choice is a little noisy.
3 seeds, paired by seed	Budget: more seeds were cut from scope.	The tests are underpowered; the paper must say so.
`T5`, not "BART/T5"	Span corruption is native to T5 and clean to reimplement.	The introduction text still needed fixing.
Report a null result honestly if it comes	All three Deltas within noise is a publishable outcome.	Only believable because budget, decoding and seeds are controlled.

10. Known weaknesses (found by reading and testing the code)

These are not hypothetical; each was confirmed in the code or in a small test. A questioner may find them. Know the answers.

1. **The in-code guard passes a `nan` loss.** `train_absa.py` refuses when `loss_after >= loss_before`, but any comparison with `nan` is false, so a `nan` slips through. Person C's check catches it. This matters most for T5 under bf16.
2. **`DAPT_BS` and `DAPT_SAVE_EVERY` are not recorded** in `dapt_report.json` or the run JSONs, and `DAPT_BATCH_TOKENS` in `config.py` is defined but never used. A leftover `DAPT_BS=8` from the smoke test would pass Gate 2 yet change the recipe (four times more optimizer steps). The log line `step N/6103` reveals the batch size: 6,103 means 32.
3. **The `transformers` version is stored but not checked.** The baselines may have used a different version from the DAPT runs; compare the field across all 24 JSONs.
4. **`corpus_source` is never written by `dapt.py`.** `train_absa.py` reads it from the report, so every `dapt` run records `corpus_source: "unknown"`. Cosmetic, but it should be known.

5. **Only one DAPT run per architecture.** The three seeds vary fine-tuning only, and `dapt.py` does not set a seed. The spread in the table does not include DAPT run-to-run variation.
6. **Test-set noise exceeds seed noise.** The test set has 1,336 rows from 500 sentences, giving about ±1 point of standard error on accuracy. Deltas under 1–2 points cannot be separated from it. A paired bootstrap over sentences using the stored `preds` and `gold` fixes this without a GPU.
7. **Six cross-architecture tests with no multiple-comparison correction.**
8. **GPT-2 prompt truncation** cuts the aspect off prompts above 124 tokens; the count comes from `scripts/check_lengths.py`.
9. **Cosmetic:** the docstring in `data_mams.py` says about 4,297 validation and test terms; 4,297 is the number of *training sentences*.

11. Questions you will probably be asked

- *Why is the learning rate different per model?* Section 9. It is the same between `base` and `dapt`, which is the contrast Δ is computed over.
- *Is the comparison between MLM, CLM and span corruption fair?* Only on tokens processed. The supervised density differs and is a stated limitation.
- *Could the T5 result be just model size?* `t5small` is the control, and the T5-base 220M versus about 125M gap is listed in Limitations.
- *How do you know DAPT worked?* The held-out Yelp language-modeling loss dropped for every model on identical corruptions (Gate 2). That shows the treatment was applied, not that it helps.
- *How do you know you did not tune on the test set?* The epoch is chosen on validation F1 and the test set is scored once, after selection, in `train_absa.py`.
- *Why can't GPT-2 and T5 just generate the label?* Because then format errors and decoding choices, not topology, would drive the differences.
- *What do your p-values mean with three seeds?* Very little on their own; the paper says the tests are underpowered and "not distinguishable" is the correct reading of close Deltas.
- *Why bf16 in DAPT but fp32 in fine-tuning?* The same choice applies to `base` and `dapt`, so it cannot create a Delta. It limits comparison with other papers.
- *What if all Deltas are about zero?* Then the paper reports that topology does not meaningfully modulate the DAPT benefit at base scale, a null result that is believable because of the controls in section 2.